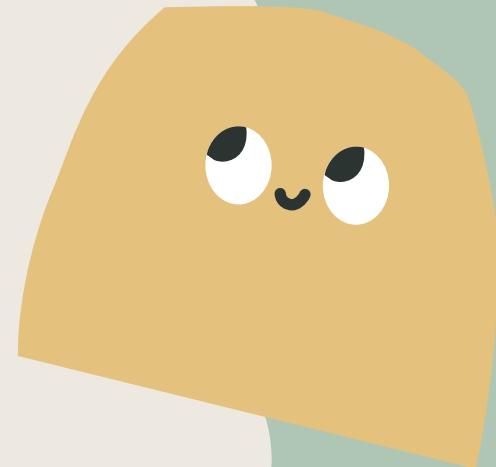
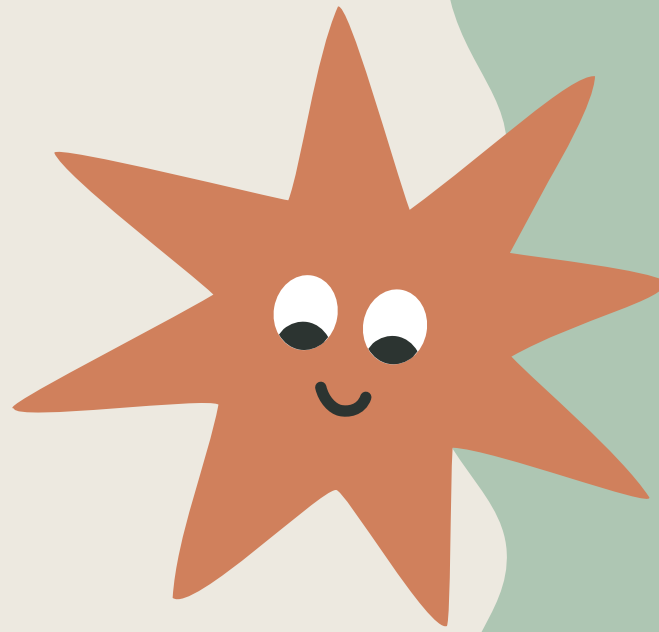


Circular Linked List

Lecture 10



Learning Outcomes (Lecture 10)

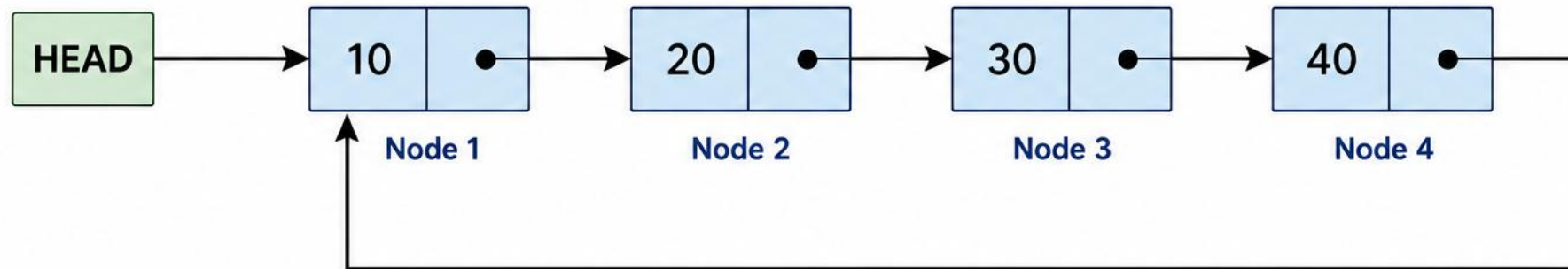
- At the end of this lecture, students will be able to:
- Explain the limitations of a singly linked list that motivate other linked-list variants
 - Explain the structure and behavior of a **Circular Singly Linked List**
 - Differentiate a circular linked list from a singly linked list
 - Perform basic traversal and insertion/deletion operations conceptually
 - Compare singly, circular, and doubly linked lists
 - Select an appropriate linked-list type for a given application

CIRCULAR LINKED LIST

DEFINITION

A circular linked list is a type of linked list in which the last node **does not** point to **NULL**, instead it points back to the first node (**head**), forming a **circle**.

STRUCTURE



Practical Applications of Circular Linked List

➤ Round-Robin Scheduling

- Operating systems use them to switch CPU time dynamically between multiple executing programs

➤ Media Playlists

- Music and video streaming apps leverage them to seamlessly loop back to the first track when a playlist ends

➤ Multiplayer Gaming

- Games use circular tracking to cycle turn progression sequentially from player to player

Practical Applications of Circular Linked List

➤ Real-Time Systems

- Circular linked list can model representing sequences like traffic light signals (Red → Green → Yellow → Red)

➤ Image Viewer / Slider

- When viewing images in a gallery, after the last image it loops back to first one

➤ Token Passing in Networks

- Circular linked list helps in managing nodes in a ring network

➤ Implementation of Circular Buffer

- Useful in buffering streaming data (e.g., audio, video streaming, real-time data)

Basic Operations on Circular Linked List

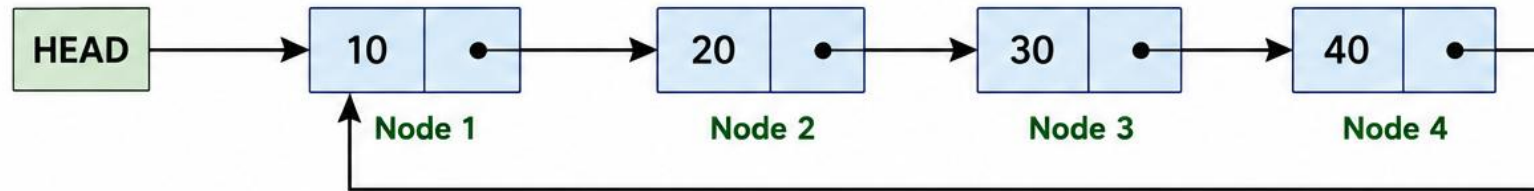
➤ Following are the basic operations supported by a Linked List:

- **Traverse** – Print all the elements of linked list one by one
- **Insertion (4 Cases)** – Adds before an element, adds after an element, adds at start, add at end of linked list
- **Deletion (3 Cases)** – Delete by value, delete from start, delete at the end
- **Search** – Searches a node by the value
- **Update** – Updates a node's value

TRAVERSE OPERATION IN CIRCULAR LINKED LIST

Traversing a circular linked list means visiting each node exactly once starting from the head and **stopping** when we reach the **head again**.

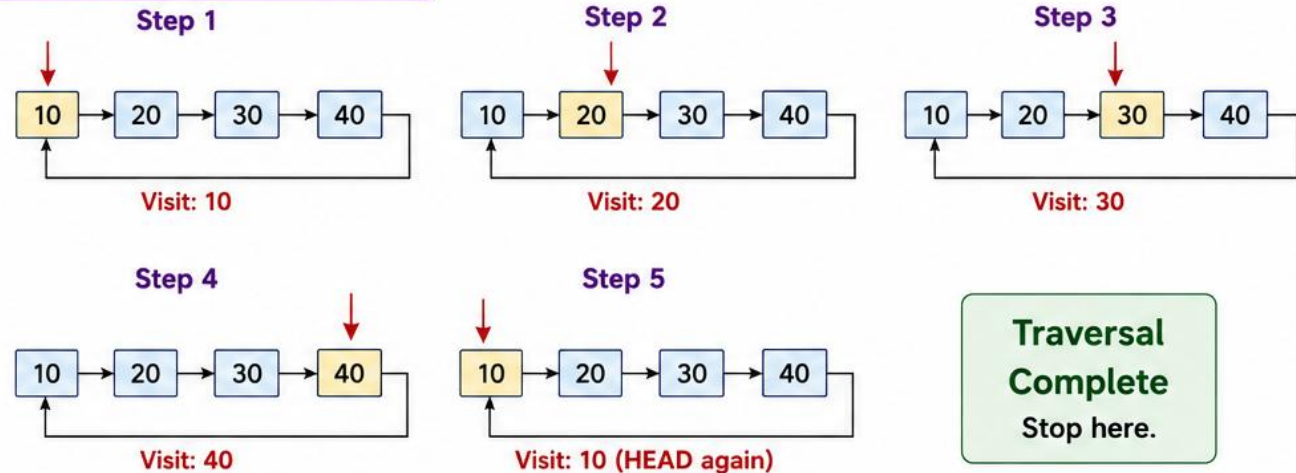
INITIAL CIRCULAR LINKED LIST



TRAVERSAL PROCESS

- 1 Start from **HEAD**.
- 2 Visit the current node and move to the next node.
- 3 Repeat Step 2 until we reach **HEAD** again.
- 4 Stop traversal.

STEP-BY-STEP TRAVERSAL



TRAVERSAL ORDER

10 → 20 → 30 → 40

IMPORTANT POINT

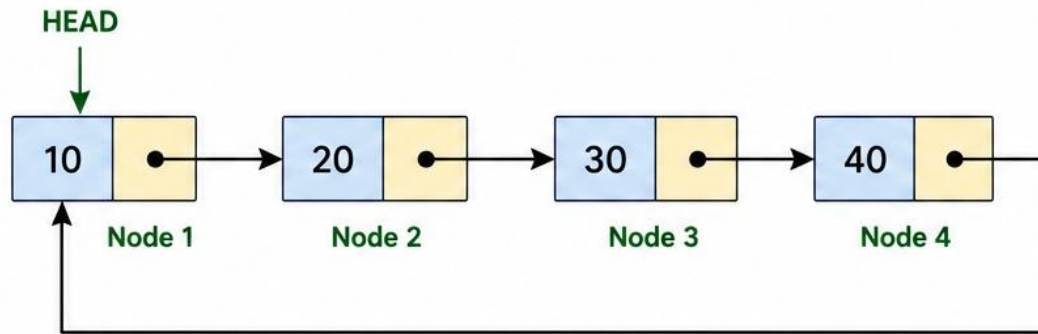
In a circular linked list, we **do not** check for **NULL**. We stop when we reach the **HEAD** again.

CIRCULAR LINKED LIST – TRAVERSE OPERATION

OPERATION

Traversal means visiting each node of the circular linked list exactly once, starting from the **head** and stopping when we reach the **head** again.

VISUAL REPRESENTATION



Traversal Order:
10 → 20 → 30 → 40

C++ CODE

```
void traverse() {  
    if (head == NULL) {  
        cout << "List is empty." << endl;  
        return;  
    }  
  
    Node* temp = head;  
    cout << "Traversal: " << endl;  
  
    do {  
        cout << temp->data << " ";  
        temp = temp->next;  
    } while (temp != head);  
  
    cout << endl;  
}
```

TIME COMPLEXITY

The loop runs exactly n times, where n is the number of nodes in the list. Therefore, the time complexity of traversal in a circular linked list is:

$O(n)$

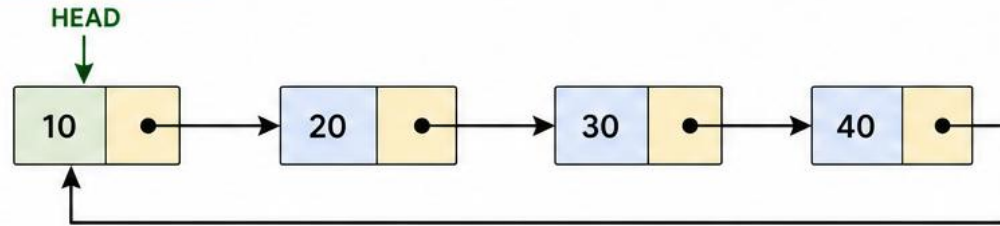
CIRCULAR LINKED LIST – INSERT AT START

OPERATION

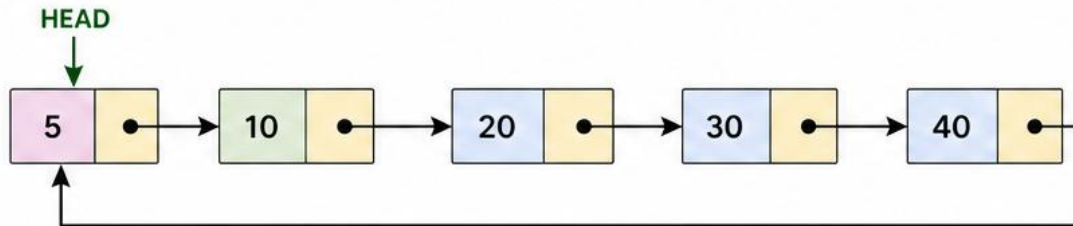
Insert at start means adding a new node before the current head.
In a circular linked list, the last node must always point to the **new head**.

VISUAL REPRESENTATION

BEFORE INSERTION



INSERT 5 AT START



STEPS

- 1 Create a new node with the given data.
- 2 If the list is empty: make new node point to itself and set head to new node.
- 3 Otherwise:
 - Find the last node (the node whose next points to head).
 - Make new node's next point to current head.
 - Make last node's next point to new node.
 - Update head to new node.

C++ CODE

```
void insertAtStart(int data) {  
    Node* newNode = new Node(data);  
    if (head == nullptr) {  
        // List is empty  
        newNode->next = newNode;  
        head = newNode;  
        return;  
    }  
  
    // Find last node  
    Node* last = head;  
    while (last->next != head)  
        last = last->next;  
  
    // Insert new node at start  
    newNode->next = head;  
    last->next = newNode;  
    head = newNode;  
}
```

TIME COMPLEXITY

We need to traverse the list to find the last node.
In the worst case, we may need to visit all n nodes.

Therefore, the time complexity is:

$O(n)$

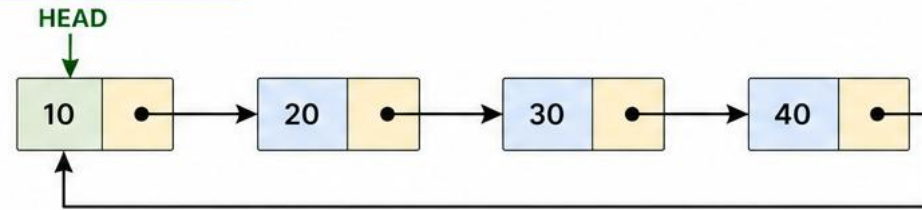
CIRCULAR LINKED LIST – INSERT AT END

OPERATION

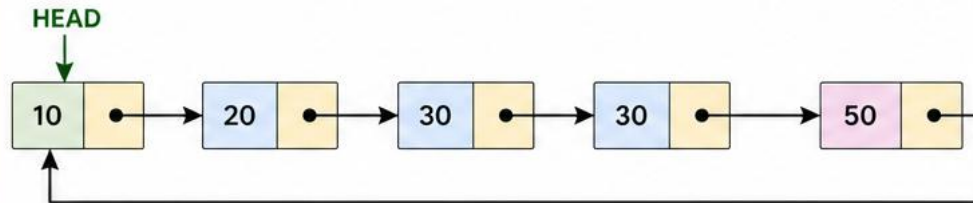
Insert at end means adding a new node after the last node so that the new last node points back to the **head**.

VISUAL REPRESENTATION

BEFORE INSERTION



INSERT 50 AT END



STEPS

- 1 Create a new node with the given data.
- 2 If the list is empty: make new node point to itself and set **head** to new node.
- 3 Otherwise:
 - Traverse the list to find the last node (the node whose next points to **head**).
 - Make last node's next point to new node.
 - Make new node's next point to **head**.
- 4 Do not change the **head**.

C++ CODE

```
void insertAtEnd(int data) {  
    Node* newNode = new Node(data);  
    if (head == nullptr) {  
        // List is empty  
        newNode->next = newNode;  
        head = newNode;  
        return;  
    }  
  
    // Find last node  
    Node* last = head;  
    while (last->next != head)  
        last = last->next;  
  
    // Insert new node at end  
    last->next = newNode; // Last node points to new node  
    newNode->next = head; // New last node points to head  
}
```

TIME COMPLEXITY

We need to traverse the list to find the last node.
In the worst case, we may need to visit all n nodes.
Therefore, the time complexity is:

$O(n)$

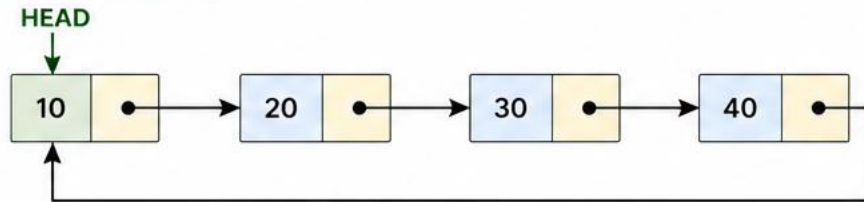
CIRCULAR LINKED LIST – DELETE AT START

OPERATION

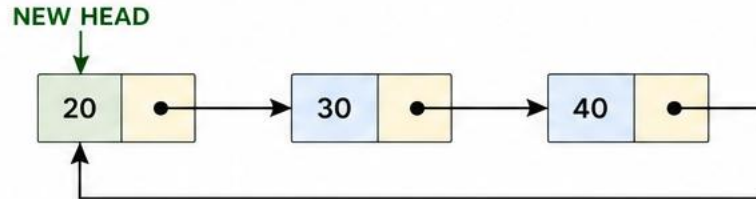
Delete at start means removing the first node (head) from a circular linked list and updating the **head**.

VISUAL REPRESENTATION

BEFORE DELETION



DELETE 10 (AT START)



STEPS

- 1 If the list is empty, do nothing.
- 2 If the list has only one node:
 - Delete it and set **head** to NULL.
 - (List becomes empty)
- 3 Otherwise:
 - Find the last node (node whose next points to **head**).
 - Move **head** to head->next (second node).
 - Make last->next point to the new **head**.
 - Delete the old **head** node.

C++ CODE

```
void deleteAtStart() {
    if (head == nullptr) {
        cout << "List is empty. Nothing to delete.\n";
        return;
    }

    if (head->next == head) {
        // Only one node
        delete head;
        head = nullptr;
        return;
    }

    // Find last node
    Node* last = head;
    while (last->next != head)
        last = last->next;

    // Store current head and move head to next node
    Node* temp = head;
    head = head->next;

    // Last node points to new head
    last->next = head;

    // Delete old head
    delete temp;
}
```

TIME COMPLEXITY

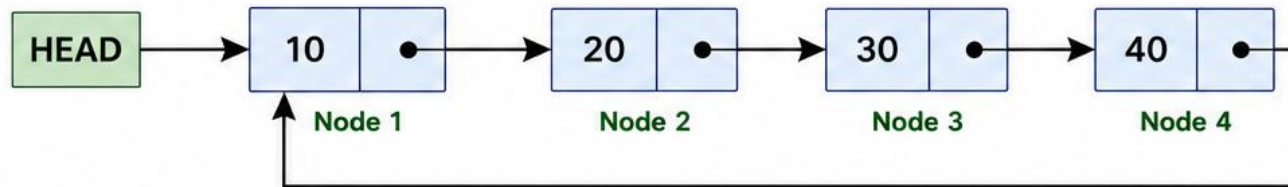
We need to traverse the list to find the last node.
In the worst case, we may need to visit all n nodes.
Therefore, the time complexity is:

$O(n)$

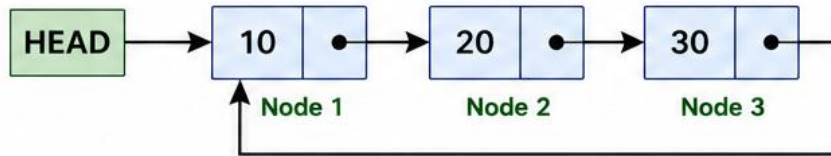
CIRCULAR LINKED LIST: DELETE AT END

Delete at End means removing the last node from a circular linked list.

INITIAL CIRCULAR LINKED LIST



AFTER DELETION (Deleted 40)



WHAT HAPPENED?

- 40 (last node) is deleted.
- 30 is the new last node.
- 30->next now points to head.

ALGORITHM (Delete at End)

1. If list is empty, return.
2. If there is only one node, delete it and set head = NULL.
3. Otherwise, traverse the list to find the second last node (node whose next is the last node).
4. Delete the last node.
5. Set second last node's next to head.

C++ CODE (Delete at End)

```
void deleteAtEnd() {  
    if (head == NULL) return;           // Empty list  
    if (head->next == head) {           // Only one node  
        delete head;  
        head = NULL;  
        return;  
    }  
    Node* temp = head;                  // Traverse to find second last node  
    while (temp->next->next != head) {  
        temp = temp->next;  
    }  
    Node* last = temp->next;             // Last node  
    temp->next = head;                  // Link second last node to head  
    delete last;                        // Delete last node  
}
```

TIME COMPLEXITY



$O(n)$

We may need to traverse all n nodes to find the second last node.
Therefore, the time complexity is $O(n)$.